T-Minus_6 Game and Software Build Specification

Give this whole document to Claude Fable 5.1. It tells Fable what to build, how the hardware and software connect, and how to prove it works.

D NOVAS · NASA Space Apps Challenge 2026, Toronto · written Nov 11, 2026

> **How to use this document (for Dhyanesh).** 1) Start a new chat with Claude Fable 5.1. 2) Attach this PDF, plus the project PDF and the GitHub repo zip (they hold the wiring and the starter code). 3) Paste the prompt below. 4) Ask Fable to build in the milestone order (Section 20) and to give you files, not snippets. 5) Run the Hardware Check screen before every demo.

Prompt to paste into Fable

```
You are the lead engineer for team D NOVAS (NASA Space Apps 2026). Read the attached
"T-Minus_6 Game and Software Build Specification" completely before writing code. Build everything
it lists: the laptop game (Python), the firmware for the three boards, the UNO Q bridge programs,
the simulator, the tests and the README. Follow the milestone order in Section 16. Every requirement
marked MUST is mandatory. Where the spec says "verify", check the current official documentation
(Arduino UNO Q / App Lab, pygame-ce) and tell me what you found. Give complete files with paths,
never fragments. After each milestone, tell me exactly how to run and test it. If anything is
ambiguous, pick the safest option, say which, and keep going. The finished game must look like a
professional, high-graphics game and run at 60 frames per second on a normal Windows laptop.
```

1. What we are building

T-Minus_6 is a game spacecraft that orbits Earth on a laptop screen, and a real four-wheel robot with the same name that copies every move on a cardboard board. One Bluetooth gamepad (Xbox style) drives both at the same moment. Debris sits on the orbit ring (real objects on the table and virtual ones on screen). A tower fires slow paper balls as real asteroids, and the same asteroid flies across the screen. A weather station warns the crew when the spacecraft enters a hot zone or a cold zone. The craft has 15 lives. An auto-pilot dodges hazards. A camera on the robot gives a first-person view.

Goal of this document: a game and software stack that, when the hardware is plugged in, **detects every device by itself, syncs the game and the robot with minimal delay, and survives a 3-minute judged demo with nothing going wrong.**

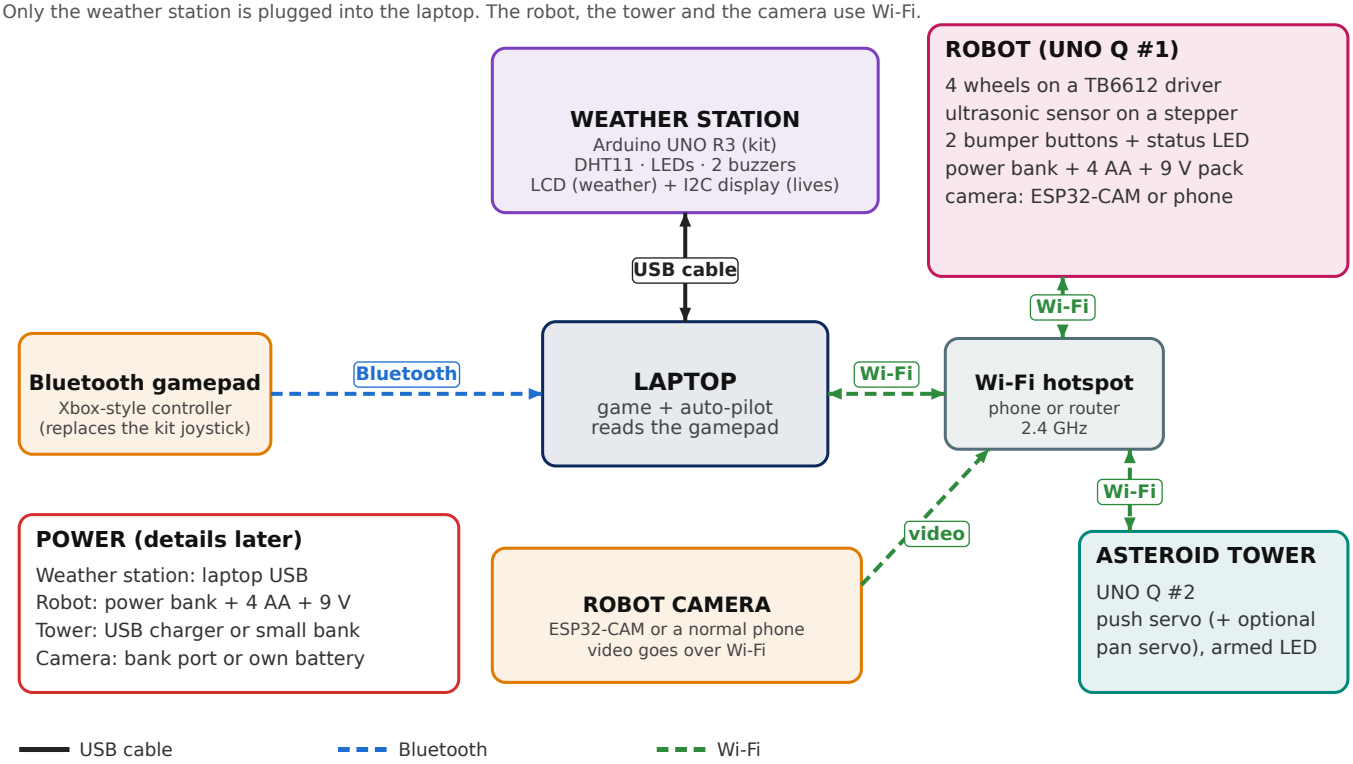
Players of the demo

- One player holds the gamepad. Judges watch the screen and the robot on the board.
- The game must also be fully playable with **no hardware** (simulation mode) so it can be built and tested anywhere.

2. The machines and how they connect

The whole hardware at a glance

Only the weather station is plugged into the laptop. The robot, the tower and the camera use Wi-Fi.



The four machines. Solid line = cable, dashed = wireless. (Same picture as in the project PDF.)

Machine	Link to laptop	Job
Laptop (Windows 10/11)	-	Runs the game, the one auto-pilot, discovery, logging. Central hub.
Bluetooth gamepad	Bluetooth (OS-paired)	Read with pygame.joystick. Left stick drives. Buttons per Section 8.
Weather station (ELEGOO UNO R3)	USB cable, serial 115200	DHT11 air sensor, 16-pin weather LCD, LEDs, two buzzers, I2C lives display (optional).
Robot T-Minus_6 (Arduino UNO Q #1)	Wi-Fi (UDP) on a phone hotspot. USB-C serial fallback	4 wheels, ultrasonic turret, bumpers, status LED.
Asteroid tower (Arduino UNO Q #2)	Wi-Fi (UDP). USB-C serial fallback	One push servo drops a paper ball down a ramp. Test button, armed LED. Optional pan servo.
Camera	Wi-Fi (HTTP stream)	ESP32-CAM (MJPEG) or a phone camera app. Never uses the control link.

Everything wireless joins one 2.4 GHz phone hotspot (name and password stored in a local, git-ignored settings file). The laptop joins the same hotspot.

Hardware facts the software must respect

- UNO R3 pins are 5 V. UNO Q pins are 3.3 V. The ultrasonic ECHO line is divided down by 1 kohm + 2 kohm.
- Robot drive: left side = 2 motors on TB6612 channel A, right side = 2 motors on channel B. Motors are 3-6 V, 200 RPM, 65 mm wheels, so full speed is about 68 cm/s. **The firmware must cap speed** (default

120 of 255) and the game must treat the robot as slow (target 10-20 cm/s).

- The UNO Q has a Linux side (Python, Wi-Fi) and a microcontroller side (Arduino sketch). They talk through the Arduino **Bridge** (verify the current API in the official UNO Q / App Lab documentation; likely Python `arduino.app_utils` on the Linux side and `Arduino_RouterBridge` in the sketch).
- Pin maps and wiring are in the project PDF and in `hardware/pin_map.md` in the repo. **Do not change pin numbers** without telling the user.

3. Technology choices (MUST)

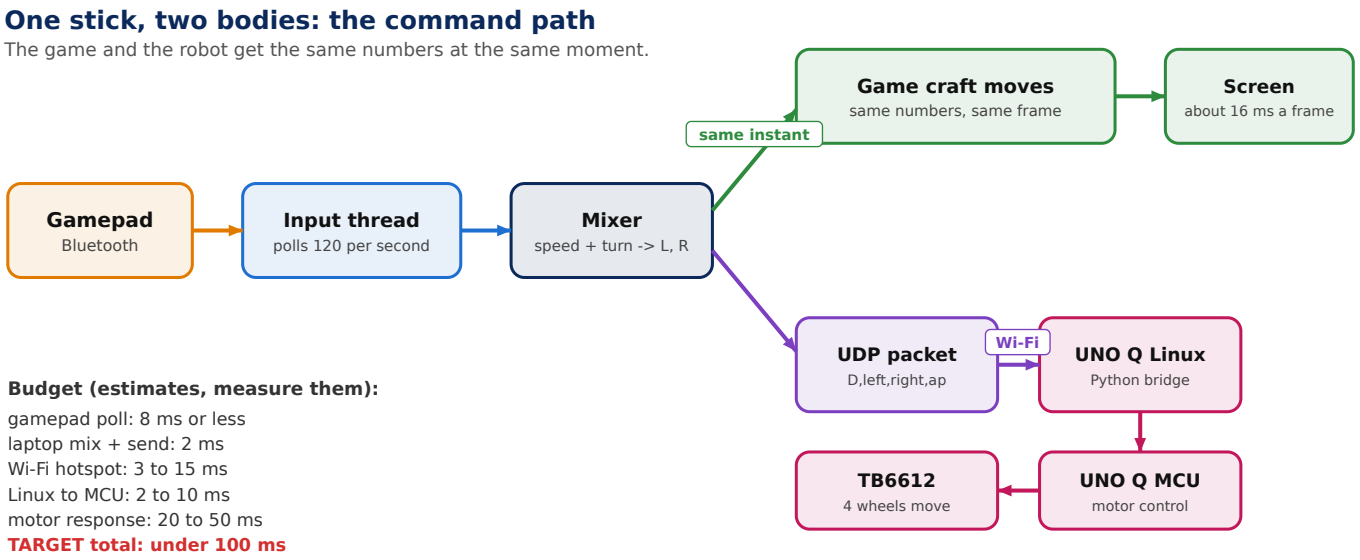
Item	Choice
Language	Python 3.11+ on Windows (keep it cross-platform where easy).
Graphics	**pygame-ce** (community edition) with numpy. 1920x1080 target, windowed + fullscreen toggle (F11), 60 fps. Optional `moderngl` post-processing for bloom and vignette with an automatic CPU fallback if it is missing. Verify pygame-ce installs with `pip install pygame-ce`.
Assets	**Everything is generated in code.** No downloaded images or sounds. Earth, stars, ship, debris, asteroids and particles are drawn procedurally. Sounds are synthesized with numpy (engine hum, beeps, alarms, impact). No copyrighted or branded material. No real people.
Concurrency	Threads for: gamepad polling, weather serial, robot link, tower link, camera. The render loop never blocks on I/O. Use thread-safe queues and a single shared state object with a lock.
Networking	UDP for control (low latency). HTTP/MJPEG for camera. Serial for the weather station. TCP is not used for control.
Config	`config.py` (committed defaults) plus `config.local.json` (git-ignored, Wi-Fi details, camera URL, overrides). Everything below that is a number is a config value with a default.
Dependencies	`pygame-ce`, `numpy`, `pyserial`, `opencv-python` (camera), optional `moderngl`. Provide `requirements.txt` and a one-command install.
Tests	`pytest` for the model, protocol parser, auto-pilot and state machine. A hardware-in-the-loop script for each device.

4. Deliverables (the file tree Fable must produce)

```
tminus6/
  README.md                      how to install, connect, run, troubleshoot
  requirements.txt
  laptop/
    main.py                      entry: --sim (no hardware), --windowed, --nohud-debug
    config.py   config.local.example.json
    core/        state.py  statemachine.py  model.py  autopilot.py  hazards.py  lives.py
                 weather.py  calibration.py  events.py  clock.py
    io/          gamepad.py  keyboard_fallback.py  discovery.py  robot_link.py  tower_link.py
                 weather_link.py  camera.py  protocol.py  sim_devices.py  recorder.py
    gfx/         renderer.py  world.py  ship.py  earth.py  starfield.py  particles.py  postfx.py
                 hud.py  widgets.py  screens.py  fonts.py  palette.py  shake.py
    audio/       synth.py  mixer.py
    tools/       hardware_check.py  latency_test.py  gamepad_test.py  calibrate_robot.py
  firmware/
    weather_station/weather_station.ino      (v2, Section 16)
    robot/robot.ino                          (MCU side)
```

```
  robot/linux_bridge/robot_bridge.py       (UNO Q Linux side: UDP <-> Bridge)
  tower/tower.ino
  tower/linux_bridge/tower_bridge.py
tests/
docs/    (wiring PDF, diagrams)
```

5. How software and hardware sync (the core of the project)



The command path. The same numbers move the game craft and the robot at the same instant.

5.1 Rules for minimum delay (MUST)

1. **Poll the gamepad at 120 Hz in its own thread** (not in the render loop). Use `pygame.event.pump()` or SDL events inside that thread, or the cross-platform `inputs` / `hid` fallback if pygame cannot be pumped off the main thread. Verify which works on Windows and say so.
2. **One mixer.** The thread produces `(speed, turn)` in -1..1 after a dead zone of 0.08 and a response curve (default expo 0.25, configurable). The same pair goes to the game craft and to `wheel_mix()` for the robot. There is no second code path.
3. **Send the robot command every 20 ms** (50 Hz) as one small UDP packet, **and immediately when the value changes** by more than a small epsilon. Never wait for an acknowledgement before the next send. No retries, no queues: a late command is worse than a new one.
4. **Failsafe on the robot:** if no `D` line arrives for 300 ms, stop all four wheels. If the laptop game crashes, the robot stops.
5. **Latest value wins.** The UNO Q Linux bridge keeps only the newest command and writes it to the microcontroller; it drops old ones.
6. **Hold the UNO Q at full Wi-Fi speed:** disable Wi-Fi power saving on the Linux side if possible (verify), use the phone hotspot at 2.4 GHz, keep video traffic modest.
7. **Measure, do not guess.** Implement a round-trip test (Section 5.3) and show the result on screen. Target one-way gamepad-to-wheel under 100 ms; stretch goal 50 ms.
8. **Screen and robot move together.** Provide `sync_delay_ms` (default 0). When greater than zero the game craft is delayed by that amount so the screen matches the robot's real motion. The calibration screen can set it automatically from the measured one-way latency. Default behaviour: the screen reacts instantly and the on-screen craft is nudged by the robot's dead-reckoned pose (Section 14.3) only for drift correction.

5.2 One movement model (MUST)

The game craft state is `(x, y, theta)` in centimetres on a 120 x 80 cm board (configurable). Per tick:

```
v     = speed * V_MAX            # cm/s, V_MAX default 15 (measure it)
omega = turn  * W_MAX            # deg/s, W_MAX default 90 (measure it)
theta += omega * dt
x += v * cos(theta) * dt ; y += v * sin(theta) * dt
left  = speed + K * turn ; right = speed - K * turn      # K default 0.6
scale both so max(|left|,|right|) <= 1 ; send int(255 * left), int(255 * right)
```

Both left wheels get `left`, both right wheels get `right` (tank steering). Provide `calibrate_robot.py`: drive 50 cm forward and turn 360 degrees, the user types the measured numbers, and `V_MAX`, `W_MAX`, `K` are written to `config.local.json`.

5.3 Latency measurement (MUST)

- Laptop sends `P,<seq>,<t_laptop_us>` every 500 ms to robot and tower. Devices answer `Q,<seq>,<t_laptop_us>` immediately. Round trip = now - t. Show the smoothed RTT and the one-way estimate (RTT / 2) in the HUD. Colour: green under 40 ms, yellow under 80, red above.
- `tools/latency_test.py`: sends 500 pings and prints min / median / 95th percentile / max, and packet loss.

6. Device discovery and connection (MUST)

The user plugs things in and powers them on; the game finds them. No typing of IP addresses in normal use.

Device	How the game detects it
Gamepad	`pygame.joystick` device-added/removed events. Show the pad name. If no pad: keyboard fallback (WASD/arrows) and a banner "Pair the controller in Bluetooth settings".
Weather station	List serial ports (`serial.tools.list_ports`). For each candidate (prefer VID 0x2341 / 0x1A86 ELEGOO CH340 clones) open at 115200, send `I?`, expect `I,weather,2` within 1.5 s. Remember the port. Re-scan every 3 s while missing. Handle unplug/replug without restarting the game.
Robot and tower	**UDP discovery.** Each UNO Q bridge program broadcasts `HELLO,robot,<fw_version>,<ip>` (or `tower`) to the subnet broadcast address, UDP port 5005, once a second. The laptop listens on port 5005, records the sender IP, and connects. The laptop also broadcasts `WHO?` once a second while a device is missing; devices reply with HELLO. Devices consider the laptop connected after the first `P` or `D` packet and treat 2 s of silence as disconnected. Manual override: `ROBOT_ADDR` and `TOWER_ADDR` in `config.local.json`.
USB-C fallback	If a UNO Q appears as a serial port answering `I?` with `I,robot,2` or `I,tower,2`, use `SerialLink` with the same line protocol. Prefer the link with the lowest RTT.
Camera	Try the configured URL (default `http://<ip>:81/stream` for ESP32-CAM; for a phone app, the URL shown in the app). Optionally probe the robot's IP neighbours. Show LIVE / NO SIGNAL. Never block the game when the camera is missing.

Status strip (MUST, on every screen)

Five dots with labels: **Pad, Robot, Tower, Station, Camera**. Green = connected and healthy, yellow = connected but RTT above 80 ms or packet loss above 5 percent, red = missing. Clicking or pressing F2 opens the **Hardware Check** screen.

Hardware Check screen (MUST)

One page that tests everything before the demo: shows each device, its IP or port, firmware version, RTT; buttons to **spin each wheel side briefly** (robot lifted), **sweep the turret**, **fire the tower once**, **flash each weather LED and buzzer**, **show hot and cold warnings on the weather LCD**, and read the gamepad axes live. A green "READY FOR DEMO" appears only when every check has passed this session.

7. Connection steps for the user (put these in README.md)

1. **Hotspot:** turn on a phone hotspot, 2.4 GHz. Note name and password. Put them in `config.local.json` (never commit them).
2. **UNO Q boards:** power each from its power bank. In Arduino App Lab (or the board's network settings) join the hotspot. Upload the sketch (MCU side) and start the bridge Python program (Linux side) so it runs on boot (verify how App Lab makes an app auto-start).
3. **Laptop:** join the same hotspot. Allow Python through Windows Firewall on **Private networks** (UDP 5005).
4. **Weather station:** plug the UNO R3 into the laptop with the USB cable. No setup.
5. **Gamepad:** pair in Windows Bluetooth settings (Xbox: hold the pair button until the light blinks fast).
6. **Camera:** ESP32-CAM: power it from the robot's power bank USB-A, join the same hotspot (Wi-Fi details are in its sketch). Phone: start the streaming app on the same hotspot, put the URL in `config.local.json`.
7. Run `python main.py`. The status strip should turn green within 10 seconds. Open Hardware Check (F2) and run every test.

README.md must also contain a **troubleshooting table** (device not found, high RTT, robot stops, wheel wrong direction, camera lag, weather LCD blank, Windows firewall).

8. Controls

Gamepad (Xbox layout)	Action
Left stick up / down	Speed forward / back (robot and game)
Left stick left / right	Turn left / right (robot and game)
Y	Toggle auto-pilot (off at start). The dodge overrides the stick only in the danger zone.
B	Toggle robot camera view (picture-in-picture / full screen)
X	Fire a test asteroid (only in practice mode)
A	Confirm / select in menus
Start	Pause. After game over: restart. Hold 2 s: re-sync the robot to its start mark and reset the game.
Back / View	Open Hardware Check
Triggers (optional)	Boost / brake scaling of speed (0.5x to 1x), configurable
Keyboard fallback	W A S D or arrows to drive, Space = auto-pilot, C = camera, Enter = confirm, Esc = pause, F2 = hardware check, F11 = fullscreen

Button numbers differ by OS and driver. Map buttons by name through a table in `config.py`, and ship `tools/gamepad_test.py` that prints axis/button numbers. Support rumble on a hit and on a warning if the pad allows it (`joystick.rumble`); ignore failures.

After game over the stick does nothing (robot and game ignore it). Only Start works.

9. Game design: it must look like a professional, high-graphic game

9.1 Look and feel

- **Theme:** deep-space, cinematic, clean sci-fi UI. Dark navy background, cyan/blue accents, orange for danger, red for heat, ice blue for cold. Consistent palette in `palette.py`.
- **Resolution:** render at 1920x1080 internally, scale to the window with smooth scaling. Support any window size. Vsync on.
- **Camera:** top-down, smooth follow is not needed (the whole board is visible), but add a slight zoom and parallax drift for depth. Subtle screen shake on hits and impacts.
- **Fonts:** a clean sans (generate with a bundled-by-code fallback; try system fonts like Segoe UI and Consolas; never crash if missing).

9.2 Visual elements (procedural, MUST)

Element	Spec
Starfield	3 parallax layers, twinkling, occasional shooting stars. Slight nebula gradient behind.

Earth	Procedural planet: layered noise for oceans and continents, clouds that slowly rotate, day/night terminator, thin glowing atmosphere ring. Radius about 15 cm equivalent on the board.
Orbit ring	Glowing dashed ring at the orbit radius (default 40 cm equivalent, a config value). Segment glow follows the craft. The **hot zone** (sunlit arc) is tinted orange-red and the **cold zone** (Earth's shadow arc) is tinted ice blue, both labelled.
T-Minus_6 craft	Sleek hand-drawn polygon craft with gradient hull, glowing cockpit, animated engine flame whose size follows speed, navigation lights. Leaves a fading trail. Flashes during the 1.5 s invulnerability.
Debris	Irregular rotating polygons with shading, matching positions in the scenario file (real ones on the table plus virtual ones). A faint red proximity ring appears when the craft is near.
Asteroids	Rotating rocky bodies with craters (procedural), glowing orange trail, a pulsing target line toward the craft before launch (Section 16).
Particles	Engine exhaust, collision sparks and fragments, hit shockwave ring, heat shimmer in the hot zone, ice crystals in the cold zone, burn-up debris on game over.
Post-processing	Bloom on bright elements, vignette, soft chromatic aberration only during damage flashes, scanline overlay for the HUD (subtle). Toggle with F3. Fallback to a cheap downscale-blur bloom if moderngl is missing.
Robot ghost	An outline marker showing the robot's estimated real pose on the board, so any drift from the on-screen craft is visible.

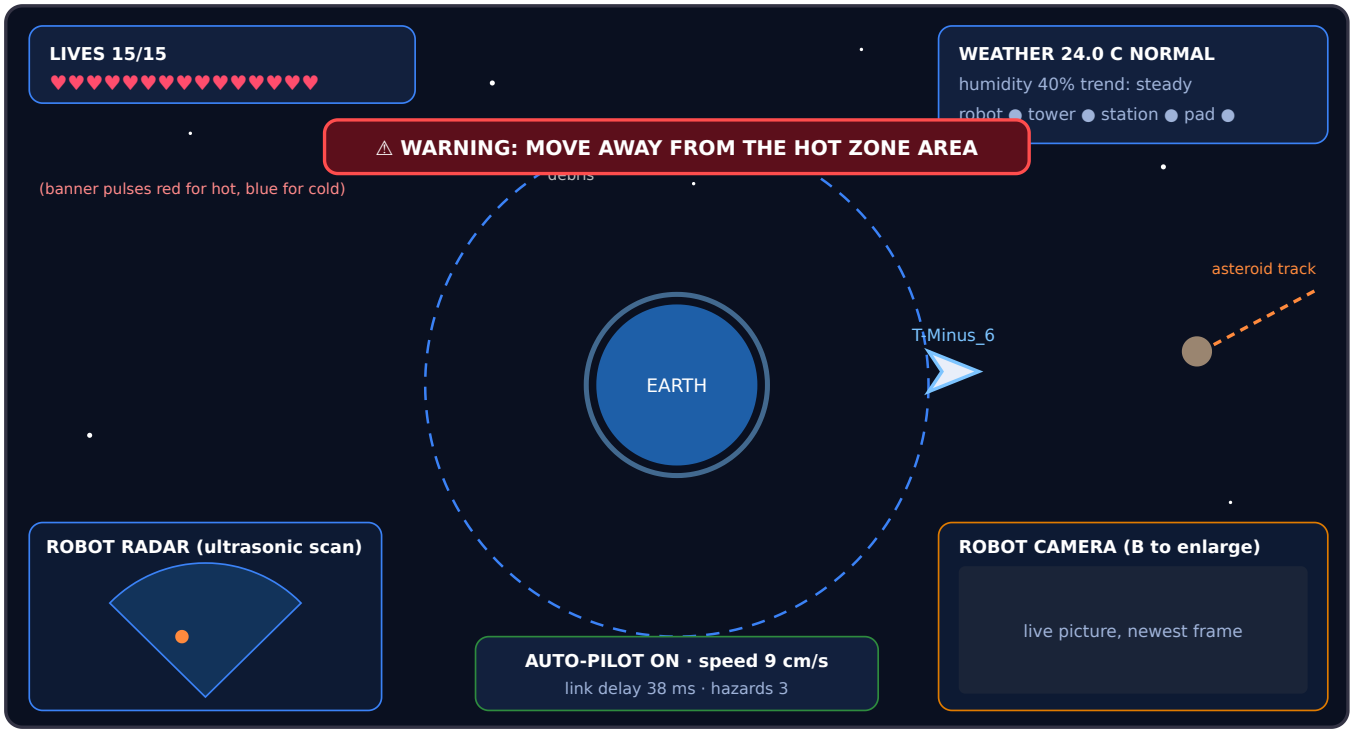
9.3 Audio (synthesized with numpy, MUST)

- Looping low engine hum whose pitch follows speed. Soft ambient space pad.
- UI clicks, connect/disconnect chimes for each device.
- Asteroid incoming alert (rising tone), impact boom, hit sound, heart-lost sound.
- Hot warning: rapid sharp beeps. Cold warning: slower low double-tone. (The physical buzzers beep too; the laptop sound is optional and can be muted with M.)
- Game over: engine dies, alarm, silence. Master volume setting.

9.4 Screen layout

Screen layout (wireframe)

Dark space theme. The game fills the whole screen; the panels float on top.



Screen layout wireframe. The game fills the screen; panels float over it.

HUD requirements (`hud.py` , `widgets.py`):

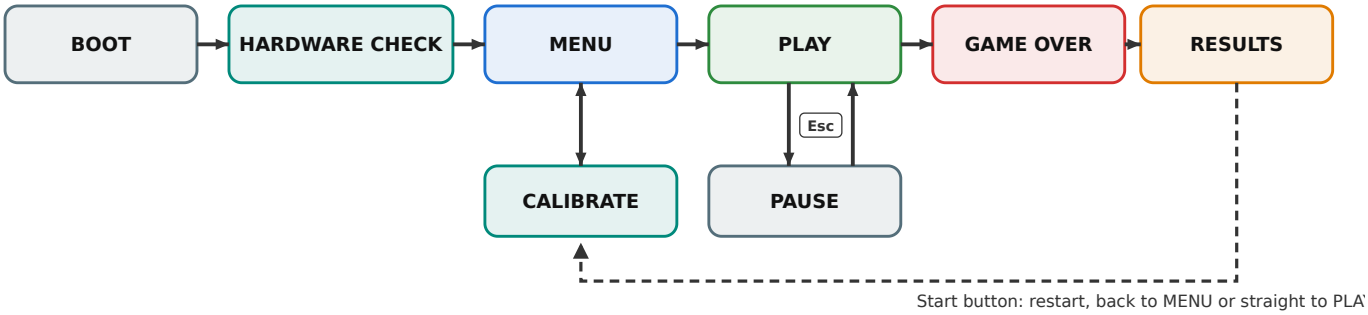
- **Lives (top left):** 15 hearts, glowing. A hit animates the last full heart cracking and fading to an outline, with a small red flash.
- **Weather panel (top right):** temperature in C, state (NORMAL / HOT / COLD), humidity, a small sparkline of the last 60 s, and the trend text (steady, rising, falling, spike).
- **Device strip:** Pad, Robot, Tower, Station, Camera dots, plus RTT in ms.
- **Warning banner (top centre):** large, pulsing, with an icon. Texts in Section 12.
- **Robot radar (bottom left):** a 90-degree fan showing the ultrasonic readings from the turret as live dots/lines; real detected debris is added to the world.
- **Camera picture-in-picture (bottom right):** live picture, newest frame, with its own LIVE/NO SIGNAL badge. B toggles full screen with the HUD drawn over it.
- **Auto-pilot panel (bottom centre):** state, speed in cm/s, nearest hazard distance, link delay.
- **Score and timer:** seconds on orbit without a hit, hazards dodged.
- Smooth number tweening, fade-ins and consistent easing everywhere.

10. Screens and state machine

Game states

Hardware keeps working in every state. Game over latches the robot until restart.

Every screen shows the device status strip (gamepad, robot, tower, weather station, camera).



Game states.

- **BOOT:** animated T-Minus_6 logo, loads fonts, synthesizes sounds, starts the device threads.
- **HARDWARE CHECK:** live device list, tests, "READY FOR DEMO" badge. Skippable. In `--sim` it is auto-green.
- **MENU:** Play, Practice (no lives lost, X fires asteroids), Calibrate, Settings (volume, post-fx, thresholds, demo mode), Quit.
- **CALIBRATE:** measures speed and turn, sets `V_MAX`, `W_MAX`, `K`, `sync_delay_ms` and the tower aim table.
- **PLAY:** the game. **PAUSE:** dims, robot stopped (send zero commands).
- **GAME OVER:** failed-orbit animation (Section 13), robot latched, results.
- **RESULTS:** time survived, hits from debris and from asteroids, dodges, best score, "Press Start".

11. Hazards, asteroids and the tower (MUST sync with hardware)

11.1 Debris

A `scenario.json` lists debris pieces (x, y, size, real: true/false). Real pieces are placed on the board to match. The robot's ultrasonic turret detects real ones (`S,angle,cm,flags`); the game converts each reading to board coordinates using the robot pose and adds or confirms a hazard. Virtual-only hazards exist in the game alone. A hit happens when the craft overlaps debris, or the robot's bumper reports `B,L` or `B,R`.

11.2 Asteroid sequence (the tower and the screen move together)

1. **Spawn plan:** an asteroid is scheduled for the **asteroid sector** of the ring (a marked arc). Only spawn while the craft is inside or heading into the sector. Minimum gap between asteroids: 6 s (configurable).
2. **Warning (1.0 s):** screen flash, alert tone, a pulsing line from the tower toward the craft's **predicted position** in about 2 s (`predict_pose(t=2.0)`).
3. **Aim (optional):** if the tower has a pan servo, send `A,<angle>` using the aim table from calibration.
4. **Fire:** send `F` to the tower. The tower releases one ball and replies `K`.
5. **Start the on-screen asteroid at the moment `K` arrives** (or 250 ms after `F` if no `K` returns, with a small "tower offline" note). Its speed equals the measured paper-ball speed (default 25 cm/s so flight time from launch to the orbit lane is 2-3 s).
6. The asteroid travels from the tower toward the craft along the planned line, with trail, rotation, sparks. It can be dodged by moving out of the lane.
7. **Hit test:** circle overlap (radius from `ASTEROID_R` and `CRAFT_R`) at the instant the asteroid crosses the lane. A paper ball cannot be sensed, so the game decides. Provide `LANE_WIDTH` as a tunable number.
8. Tower disconnected: the game can still spawn virtual asteroids (visual only) and shows "TOWER OFFLINE".

Safety: never send `F` unless the game state is PLAY or Practice, the tower is armed, and at least 2 s have passed since the previous `F`. On game over send `X`; on restart send `R`.

11.3 Hit rules

- Each hit takes exactly one life. After a hit the craft is invulnerable for 1.5 s (flashing); further collisions in that time do nothing.
- Every hit also: shakes the screen, plays the hit sound, rumbles the pad, sends `H,<lives>` to the weather station (yellow hit LED flashes 1 s and the I2C display updates).

12. Weather: hot and cold warnings (MUST)

This is a signature feature. It must work from **real air** (hair dryer / ice pack at the DHT11) **and** from **game events** (the craft entering the hot or cold zone, or a scripted event).

12.1 States and thresholds

State	Trigger	Physical response (weather station)
NORMAL	Temperature 18-30 C (config) and the craft is outside both zones	Green LED solid, no sound

HOT	Sensor above 30 C, **or** the craft is inside the hot zone, **or** a scripted hot event	Red LED blinks about 5 times a second; **active buzzer** beeps fast; LCD shows the hot warning
COLD	Sensor below 18 C, **or** the craft is inside the cold zone, **or** a scripted cold event	Blue LED blinks about 5 times a second; **passive buzzer** low tone, beeping fast; LCD shows the cold warning

Hysteresis of 1 C on the sensor. If the sensor and the game disagree, the more severe state shows. **Note for the build:** the user's early message said "red LED" for cold too; the hardware has a separate **blue** LED for cold. Keep blue for cold (one config value, `COLD_LED_COLOR`, documents it).

12.2 Warning texts

Where	Hot	Cold
Game banner (big, pulsing)	**WARNING: MOVE AWAY FROM THE HOT ZONE AREA** sub-line: T-Minus_6 is in a hot zone. Hull temperature rising.	**WARNING: MOVE AWAY FROM THE COLD ZONE AREA** sub-line: T-Minus_6 is in a cold zone. Hull temperature falling.
Banner colours	Red / orange, heat shimmer	Ice blue, frost edge
Weather LCD (16 chars per line)	Line 1: `!! HOT ZONE !!` Line 2: `MOVE AWAY NOW!`	Line 1: `!! COLD ZONE !!` Line 2: `MOVE AWAY NOW!`
Voice/tone cue (laptop, optional)	Rapid beeps	Slow double tone

12.3 Temperature changes also trigger warnings in the game (MUST)

- **State change:** every switch NORMAL to HOT, NORMAL to COLD, and back, shows a toast ("Temperature back to normal") and logs an event.
- **Spike warning:** if the temperature changes by 3 C or more within 10 s, show a yellow banner **TEMPERATURE SPIKE DETECTED** even if still within limits.
- **Trend arrow** in the weather panel (rising, falling, steady) from a 10 s window.
- **Game effects:** in HOT the screen gets a red vignette and heat shimmer; in COLD a frost vignette. Optional gameplay effect (setting, default on in demo): the craft's turn rate drops 20 percent in a danger zone. Never reduce lives because of weather.
- **Log:** time, temperature, humidity, state every 5 s (CSV in `logs/`).

12.4 Hot and cold zones in the game

The ring has a sunlit arc (hot, default 90 degrees) and a shadow arc (cold, default 90 degrees). When the craft's angle is inside an arc, the laptop sends `Z,2` (hot) or `Z,1` (cold); outside, `Z,0`. The zone state is sent only on change (plus every 2 s as a keep-alive). A scripted demo mode can force a zone with a key (H hot, K cold, N normal) for rehearsal.

12.5 Real sensor path

The station reports `W,temp,hum,state` every 2 s. State 1 or 2 from the sensor triggers the same warning in the game within one report (maximum 2 s). For a snappier demo, the station also sends an immediate `W` line on every state change.

13. Lives, game over, failed orbit

- 15 lives. Hearts shown on the HUD and (when the I2C display is connected) on the lives display: 15 hearts on line 1, `T-Minus_6 L15/15` on line 2.
- At 0 lives: freeze input (except Start), send `G` to the weather station, `X` to the robot (5 per second until it confirms) and `X` to the tower.
- **Failed-orbit animation (about 8 s):** engine flame flickers and dies, red warning lights, text MAIN ENGINE FAILURE; craft tumbles, drifts off the ring, the dashed ring breaks into pieces; gravity pulls it toward Earth with a spark trail, it glows orange, breaks into fragments and burns up at the atmosphere; fade to ORBIT FAILED and "Signal lost at T+ mm:ss"; low alarm, then silence.
- Restart (Start): reset to 15 lives, send `R` to the robot, station and tower, return to MENU or straight to PLAY (setting). Remind the user to put the robot on its start mark.

14. Auto-pilot

14.1 Behaviour

Situation	Action
No hazard, no stick	Hold the orbit radius and fly around Earth
No hazard, stick moved	Stick is a nudge (0.3 weight)
Hazard in warning zone (40 cm)	Slow to 70 percent, turn toward the clearer side
Hazard in danger zone (25 cm)	Dodge overrides the stick: speed 40 percent, hard turn until clear, then return to the lane
Anything within 8 cm ahead	Robot firmware stops forward motion by itself, even with auto-pilot off

14.2 Algorithm

1. Build the hazard list (debris, detected objects, asteroids) with position, velocity, radius.
2. Predict each hazard 3 s ahead (debris static, asteroids on their launch line).
3. If the predicted craft path comes within the danger distance, pick the side with the larger gap, steer and slow.
4. After about 1 s of clear path, ease back to the lane (smooth, no oscillation).

One brain, two bodies: there is a single auto-pilot on the laptop. Its (speed, turn) goes to the same mixer as the stick (Section 5), so the game and the robot dodge identically. Show the planned path as a faint line on screen.

14.3 Robot pose and drift

Keep a dead-reckoned pose for the robot from its commands (and the same model as the game craft). Show it as the robot ghost. A **sync** action (hold Start 2 s) puts both at the start mark. Provide `DRIFT_CORRECTION` as an optional nudge toward the ghost when the robot reports bumper or wall hits (a wall seen by the ultrasonic sensor at a known distance can correct the position estimate; implement if time allows).

15. Robot camera

- A thread reads an MJPEG stream (`cv2.VideoCapture` or a raw HTTP parser) and keeps **only the newest frame**.
- Draw as picture-in-picture (bottom right); B toggles full screen with the HUD drawn over it.
- Auto-reconnect, show LIVE and the measured frame age (ms), NO SIGNAL when older than 2 s.
- Target picture 320x240 to 640x480 at 15-20 fps. Camera load must never reduce the control link rate.
- Never record or save video of people. Add a setting "privacy: do not save frames" (default on).

16. Protocol and firmware specification

16.1 Line protocol v2 (MUST)

Text lines ending in `\n`, comma separated, ASCII, max 64 characters. Same lines over UDP (one line per datagram), USB serial and the Bridge. Unknown lines are ignored. Numbers are integers unless stated.

Direction	Line	Meaning
Any to laptop	`HELLO,role,fwver,ip`	Discovery announcement once a second (role = robot, tower, weather)
Laptop to any	`WHO?` / `I?`	Ask who is there. Reply `I,role,2`
Laptop to any	`P,seq,t_us`	Ping. Reply `Q,seq,t_us` immediately
Laptop to robot	`D,left,right,ap`	Wheel speeds -255..255, auto-pilot flag. 50 per second plus on change
Laptop to robot	`X` / `R`	Game-over latch (stop, ignore D, park turret, fast LED blink) / release
Robot to laptop	`S,angle,cm,flags`	Turret scan reading; flags bit0 = front stop active. About 15 per second
Robot to laptop	`B,L` / `B,R`	Bumper hit
Laptop to tower	`F` / `A,angle` / `X` / `R`	Fire one ball / aim (if pan servo) / disarm / re-arm
Tower to laptop	`K`	Ball released
Weather to laptop	`W,temp,hum,state`	Reading every 2 s and immediately on state change. temp has one decimal. state 0 normal, 1 cold, 2 hot (effective state)
Laptop to weather	`Z,state`	Game zone state 0/1/2 (hot or cold zone). Combined with the sensor (the more severe wins)
Laptop to weather	`V,temp`	Optional virtual temperature (simulation mode); `R` clears it
Laptop to weather	`H,lives` / `G` / `R`	Lives (hit LED flash if lower) / game over / reset to 15

16.2 Weather station firmware v2 (UNO R3) (MUST)

Start from `firmware/weather_station/weather_station.ino` in the repo. Required changes:

- Add `I?` reply `I,weather,2`, and send `W` immediately on any state change.
- Add `Z,state`. Effective state = max severity of (sensor state with hysteresis, Z state). Keep `V` as an override for tests.
- LCD texts per Section 12.2: hot `!! HOT ZONE !!` / `MOVE AWAY NOW!`, cold `!! COLD ZONE !!` / `MOVE AWAY NOW!`. Normal: `T 24.0C NORMAL` / `H 40%`. Game over: `SIGNAL LOST` / `GAME OVER`.
- Blink period 100 ms (about 5 per second) for hot and cold. Hot: red LED + active buzzer on D6. Cold: blue LED + passive buzzer tone 400 Hz on D7. Normal: green LED solid.
- Optional I2C lives display (address 0x27 or 0x3F) behind `#define HAS_LIVES_LCD 1` (default 0 until the part arrives): 15 custom hearts on line 1, `T-Minus_6 L15/15` on line 2.
- Pins (do not change): DHT11 D2; LEDs green D3, blue D4, red D5, yellow hit A0; active buzzer via PN2222 base D6; passive buzzer D7; LCD RS D8, E D9, D4-D7 on D10-D13; I2C on A4/A5. No `delay()` anywhere.

16.3 Robot (UNO Q #1)

- **MCU sketch** (`robot.ino`, repo starter): TB6612 pins PWMA D5, AIN1 D4, AIN2 D2, PWMB D6, BIN1 D7, BIN2 D8; turret stepper on D9-D12 using AccelStepper HALF4WIRE order 9, 11, 10, 12; HC-SR04 TRIG A0 and ECHO A1 (divided); bumpers A2/A3 (pull-up); status LED D13. Speed cap 120, command timeout 300 ms, 8 cm forward stop, sweep -45 to +45 degrees in 5-degree steps, game-over latch. **Replace the blocking `pulseIn`** with a non-blocking echo measurement (interrupt or timed state machine) so driving never waits on sensing.
- **Linux bridge** (`robot_bridge.py`): runs on boot on the UNO Q. Listens on UDP 5005 for lines from the laptop, forwards `D`, `X`, `R` to the MCU through the Bridge (keep only the newest `D`), answers `P` with `Q`, broadcasts `HELLO,robot,2,<ip>`, and forwards the MCU's `S` and `B` lines back to the laptop's last-seen address. Verify the Bridge API and how to auto-start an App Lab app; if the Bridge is awkward, use the USB-C serial fallback and say so.
- All behaviours must keep working if the Wi-Fi link drops: wheels stop, turret parks, LED blinks slowly.

16.4 Tower (UNO Q #2)

- **MCU sketch** (`tower.ino`): push servo D9, pan servo D10 (`HAS_PAN` flag), armed LED D13, test button D2 (pull-up). `F` releases one ball (servo to PUSH_DEG for PUSH_MS then back to REST_DEG) and replies `K`; `X` disarms (LED off), `R` re-arms. Ignore `F` when disarmed or within 1.5 s of the last fire. Make the three numbers (REST_DEG, PUSH_DEG, PUSH_MS) easy to tune from the laptop with a tool command `T,rest,push,ms` (not stored on the board, resent at connect).
- **Linux bridge** (`tower_bridge.py`): same pattern as the robot.

16.5 Version handshake

All devices report firmware version 2 in `HELLO` / `I`. If a device reports a different version, show a yellow "firmware mismatch" in the status strip and the Hardware Check.

17. Simulation mode (MUST)

`python main.py --sim` starts the whole game with **software versions of every device** (`io/sim_devices.py`): a virtual gamepad (keyboard), a virtual robot that integrates the same model with realistic delay (configurable, default 40 ms) and generates ultrasonic readings from the scenario, a virtual tower that answers `K` after 120 ms, a virtual weather station with keys to force hot, cold and normal, and a virtual camera (a rendered first-person view). The game must be fully playable in sim mode. The sim devices speak the same protocol over loopback so the real network code is exercised. Mixed mode (some real, some simulated) is allowed with flags like `--sim-robot`.

18. Reliability and error handling (MUST)

- No unhandled exceptions in threads. Each device thread catches errors, logs, marks the device red and retries with backoff (0.5 s to 5 s).
- Unplugging or powering off any device mid-game never crashes the game. The robot stops (firmware failsafe). The game shows the device red and keeps playing.
- Reconnect silently when the device returns. Re-send the current state (lives, zone, latch) on reconnect.
- Rate-limit all sends. Never block the render thread. Never sleep in the main loop.
- Log everything useful (device events, RTT, errors, game events) to `logs/YYYYMMDD_HHMMSS.log` and rotate. Include a `--debug` overlay (F4) with FPS, frame time, thread status, packet rates, latency.

- **Emergency stop:** key Space+Esc or the gamepad Back+Start held 1 s sends `X` to the robot and tower and pauses the game.
- Safe defaults when config is missing or invalid; print a clear message, never a traceback to the user.

19. Performance targets and acceptance tests

Target	Value
Frame rate	60 fps at 1080p on a normal laptop; frame time under 16 ms 99 percent of the time; graceful quality drop (post-fx off) below 45 fps
Gamepad to wheel (one way)	Under 100 ms median; stretch 50 ms. Report the 95th percentile.
Failsafe	Robot stops within 300 ms of the last command; stops within 300 ms of game over
Device detection	Within 10 s of power-on; reconnect within 5 s of return
Camera frame age	Under 300 ms typical
Memory/CPU	Stable over a 30-minute run (no growth in memory, no thread leaks)

Acceptance checklist (Fable must provide scripts or steps for each)

- ☐ **Sim run:** `--sim` plays a full round: menu, play, asteroid, hit, hot warning, cold warning, game over, restart.
- ☐ **Pad:** left on the stick turns the craft left on screen and the robot left on the table, 20 times in a row.
- ☐ **Latency:** `latency_test.py` reports median under 100 ms and under 2 percent loss.
- ☐ **Detect:** power on robot, tower, station and camera in any order; all dots turn green within 10 s without typing an address.
- ☐ **Unplug test:** pull the station USB, kill the robot's power, close the hotspot: no crash; robot stops; devices return when restored.
- ☐ **Hot air:** hair dryer at the DHT11: game banner "MOVE AWAY FROM THE HOT ZONE AREA", red LED blinks, active buzzer beeps, LCD shows hot text, within 3 seconds of the state change.
- ☐ **Cold air:** ice pack: cold banner, blue LED, passive buzzer, cold LCD text.
- ☐ **Zones:** flying into the hot arc and the cold arc triggers the same warnings with no sensor change.
- ☐ **Spike:** a 3 C change in 10 s shows TEMPERATURE SPIKE DETECTED.
- ☐ **Asteroid:** fire from the game: tower servo drops one ball and the on-screen asteroid starts within 250 ms of the release; no fire when disarmed or after game over.
- ☐ **Hits:** bumper press, debris overlap and asteroid overlap each take exactly one heart; none inside the 1.5 s protection.
- ☐ **Game over:** robot latched within 300 ms, stick ignored, animation plays, Start restores 15 hearts and unlatches.
- ☐ **Camera:** picture appears, B toggles full screen; unplugging the camera shows NO SIGNAL without stutter.
- ☐ **Soak:** three full demos in a row without restarting the program.

20. Milestone order (build in this order)

#	Milestone	Done when

1	Skeleton, config, logging, state machine, `--sim` framework	Window opens, states switch, devices simulated
2	Gamepad thread, mixer, movement model, craft on screen	Craft flies smoothly at 60 fps with a real pad
3	Graphics pass 1: starfield, Earth, ring, craft, engine, trail, HUD shell	Looks like a polished game
4	Robot link + firmware + bridge, discovery, latency test	Robot follows the stick; RTT shown
5	Weather station firmware v2 + link, warnings, zones, spike	Hot/cold tests pass
6	Hazards, debris, scan to world, lives, hits, invulnerability	Hit rules pass
7	Tower link + firmware + asteroid sequence	Asteroid synced with the ball
8	Auto-pilot, robot ghost, calibration screen	Dodge 10 of 10 in sim
9	Camera, picture-in-picture, full-screen view	Live picture, reconnects
10	Game over animation, results, audio, post-fx, polish, haptics	Demo looks and sounds professional
11	Hardware Check screen, README, troubleshooting, tests, soak	Acceptance checklist all green

21. Demo script the game must support (about 3 minutes)

1. Start: all five status dots green; READY FOR DEMO.
2. Manual run: steer by gamepad; screen and robot move together.
3. Asteroid launched; player dodges; then misses one to show a hit and a heart cracking.
4. Press Y: auto-pilot dodges debris and the next asteroid on screen and on the table.
5. Blow hot air: banner, red LED, buzzer, LCD. Cool it: cold warning. Back to green.
6. B: show the robot camera view.
7. Run lives to zero: failed-orbit animation, robot stops. Start to restart.

Add a **demo mode** setting with a scripted sequence of events (timed hot/cold zone, scheduled asteroids) so a rehearsal is repeatable.

22. Assumptions and things Fable must verify

Unverified item	What to do
UNO Q Bridge API and App Lab auto-start	Check the official docs; implement; otherwise use the USB-C serial fallback
UNO Q pins, PWM pins, 3.3 V logic and Wi-Fi power saving	Check the board's pinout; tell the user if any pin in this spec cannot do PWM
Gamepad polling from a thread on Windows	Test; use an alternative input library if needed

Which Wi-Fi band the UNO Q supports	Check; default to a 2.4 GHz hotspot
ESP32-CAM stream URL and port	Default `:81/stream` (CameraWebServer example); make it configurable
I2C display address (0x27 or 0x3F)	Include an I2C scanner and auto-try both
Real robot speed and turn rate	Use the calibration screen; defaults are placeholders

Hardware wiring, the parts list and the build guide are in the separate project PDF. This document covers software and the hardware-software connection only. Pins and protocol lines here match the repo starter code; if Fable changes any, it must update the README and the docs.